

数据结构与算法 实验报告

第一次



姓名 陈宇昂

班级 数学强基 2202 班

学号 2224110096

电话 **

Email yuang.chen2024@outlook.com

日期 2024-10-28

目录

任务 1: 证明	3
任务 2: 设计算法要多思考	4
一、 题目	4
二、 算法设计	4
三、 算法复杂度分析	4
任务 3: 排序算法的实现	8
一、 题目	8
二、 算法设计	8
三、 主干代码说明	9
(一) 插入排序 (Insertion)	9
(二) 选择排序 (Selection)	9
(三) 快速排序 (Quicksort)	9
(四) 归并排序 (Mergesort)	10
(五) 希尔排序 1 (Shell)	11
(六) 希尔排序 2 (Hibbard)	11
(七) 希尔排序 3 (Knuth)	12
四、 运行结果展示	12
任务 4: 排序算法性能测试和比较	14
一、 题目	14
二、 算法设计	14
三、 主干代码说明	14
四、 运行结果展示	16
(一) 均匀分布的随机数据	16
(二) 正序数据	16
(三) 逆序数据	16
五、 对结果的分析	17
(一) 从一个算法的运行时间变化趋势和数据规模的变化角度	17
(二) 从同样的数据规模和相同数据分布下不同算法的时间相对差异	17
六、 总结与收获	18
任务 5: 快速排序的再探讨与应用	19
一、 题目	19
二、 数据结构设计	19
三、 算法设计	19
(一) 探索递归深度与数据规模的关系	19
(二) 优化快速排序 (Optimized QuickSort)	19
(三) 螺丝螺母匹配问题	20
四、 主干代码说明	20
五、 运行结果展示	25
六、 总结与收获	27

任务 1：证明

1) 使用 O 、 Ω 和 Θ 的定义，证明下面每一个等式：

- a) $2\sqrt{n} + 6 = O(\sqrt{n})$
- b) $n^2 = \Omega(n)$
- c) $\log_2(n) = \Theta(\ln(n))$
- d) $4^n \neq O(2^n)$

证明：a) 令 $T(n) = 2\sqrt{n} + 6$ ，对于 $\forall n > 36$, $T(n) \leq 3\sqrt{n}$ ，于是 $T(n) = 2\sqrt{n} + 6 = O(\sqrt{n})$

b) 令 $T(n) = n^2$ ，对于 $\forall n > 2$, $T(n) > n$ ，于是 $T(n) = n^2 = \Omega(n)$

c) 令 $T(n) = \log_2(n)$ ，由换底公式 $\log_2(n) = \frac{\ln(n)}{\ln(2)}$ ，令 $k = \frac{1}{\ln(2)} > 0$

于是 $T(n) = k \ln(n)$, $\forall n > 1$, $k \ln(n) \leq T(n) \leq k \ln(n)$

故 $T(n) = \log_2(n) = \Theta(\ln(n))$

d) 反证法：令 $T(n) = 4^n$ ，若存在常数 c, N_0 ，满足 $c > 0, N_0 \in \mathbb{N}$ ，

有：对于 $\forall n > N_0$, $4^n \leq c \cdot 2^n$ 恒成立

但 $4^n \leq c \cdot 2^n \Leftrightarrow 2^n \leq c \Leftrightarrow n \leq \log_2 c$ ，于是取 n 充分大即可导致原命题矛盾

于是 $T(n) = 4^n \neq O(2^n)$

2) 使用数学归纳法证明 $T(n) = O(n^2)$ 。 $T(n)$ 的定义式如下：

$$T(n) = \begin{cases} T(n-1) + 2n, & n > 1 \\ 3, & n = 1 \end{cases}$$

证明：对 n 归纳。

1) 当 $n = 1$ 时， $T(1) = 3 = O(1)$ ，命题成立。

2) 假设当 $n = k$ 时，有

$$T(k) = O(k^2) \leq ck^2$$

其中 c 为一非负常数，则当 $n = k + 1$ 时，

$$\begin{aligned} T(k+1) &= T(k) + 2(k+1) \leq ck^2 + 2k + 2 \\ &\leq C(k^2 + 2k + 1) + 1 \\ &\leq (C+1)(k^2 + 2k + 1) \\ &= (C+1)(k+1)^2 \end{aligned}$$

其中 $C = \max\{1, c\}$

于是， $T(k+1) = O((k+1)^2)$ 。故当 $n = k + 1$ 时，原命题仍成立。

由数学归纳法的原理，原命题得证。

任务 2：设计算法要多思考

一、题目

已知平面上有 n 个点，设计一个算法求平面内两点距离最小的两个点。朴素的算法是通过计算任意两个点之间的距离，经过统计给出距离最短的两个点，这时的时间复杂度是 $O(n^2)$ ，现要求完成的算法时间复杂度必须满足 $O(n \log n)$ 。（提示：使用分治算法设计思想）

不需要实现代码，但必须要给出算法描述，并给出算法的时间复杂度分析过程。

二、算法设计

1. 排序：将点集 P 按照横坐标 x 排序。如果横坐标相同，则按照纵坐标 y 排序。
2. 分治：定义一个递归函数 $\text{ClosestPair}(P)$
 - (a) 若 $|P| \leq 3$ ：直接计算并返回最近点对的距离。
 - (b) 否则：
 - i. 将 P 拆分为两个子集 P_1 和 P_2 ，每个子集大小约为 $n/2$ 。
 - ii. 递归调用 $\text{ClosestPair}(P_1)$ 和 $\text{ClosestPair}(P_2)$ ，分别求得最近点对的距离 d_1 和 d_2 。
 - iii. 设 $d = \min(d_1, d_2)$ 。
3. 合并
 - (a) 构建集合 B ：将所有横坐标与分界点 x_m 的差小于 d 的点放入集合 B 。
 - (b) 将集合 B 中的点按照纵坐标 y 排序。
 - (c) 对于集合 B 中的每个点 p_i ：
 - i. 设 $C(p_i) = \{p_j \mid p_j \in B, y_j \leq y_i \text{ and } y_i - y_j < d\}$ 。
 - ii. 只考虑 $C(p_i)$ 中的点 p_j （满足纵坐标条件）：
 - 计算 p_i 和 p_j 的距离，如果小于 d ，则更新 d 。
4. 返回结果：返回最近点对的距离 d 。

三、算法复杂度分析

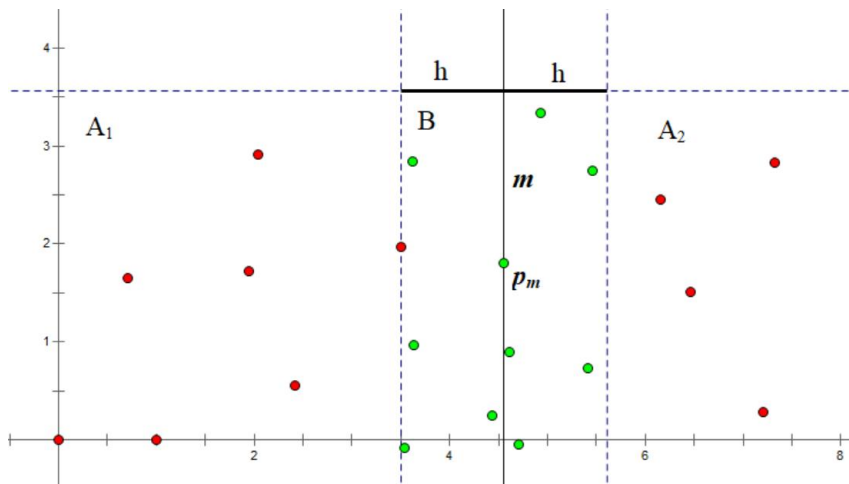
我们先将所有点按照 x_i 为第一关键字、 y_i 为第二关键字排序，并以点 p_m ($m = \lfloor \frac{n}{2} \rfloor$) 为分界点，拆分点集为 A_1, A_2 ：

$$A_1 = \{p_i \mid i = 0 \dots m\}$$

$$A_2 = \{p_i \mid i = m + 1 \dots n - 1\}$$

并递归下去，求出两点集各自内部的最近点对，设距离为 h_1, h_2 ，取较小值设为 h 。

我们寻找这样的一组点对，其中一个属于 A_1 ，另一个属于 A_2 ，且二者距离小于 h 。



将所有横坐标与 x_m 的差小于 h 的点放入集合 B ：

$$B = \{p_i \mid |x_i - x_m| < h\}$$

结合图像，直线 m 将点分成了两部分。 m 左侧为 A_1 点集，右侧为 A_2 点集。再根据

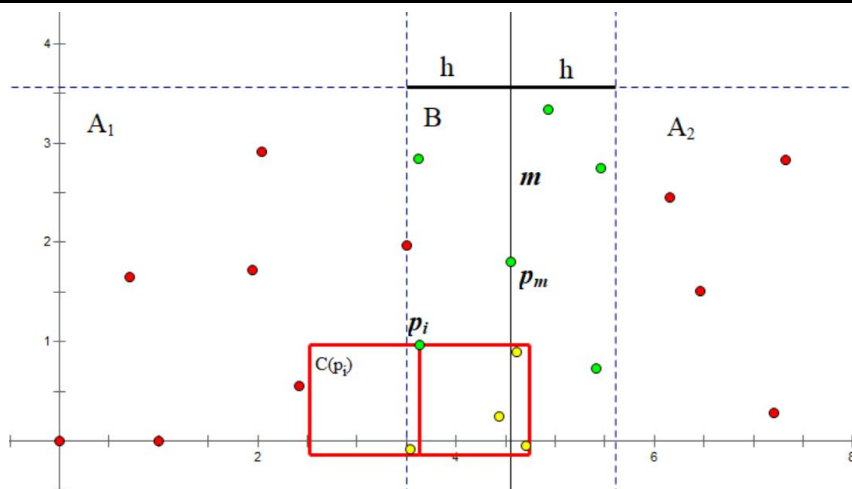
$$B = \{p_i \mid |x_i - x_m| < h\}$$

的规则，得到绿色点组成的 B 点集。

对于 B 中的每个点 p_i ，我们当前目标是找到一个同样在 B 中、且到其距离小于 h 的点。为了避免两个点之间互相考虑，我们只考虑那些纵坐标小于 y_i 的点。显然对于一个合法的点 p_j ， $y_i - y_j$ 必须小于 h 。于是我们获得了一个集合 $C(p_i)$ ：

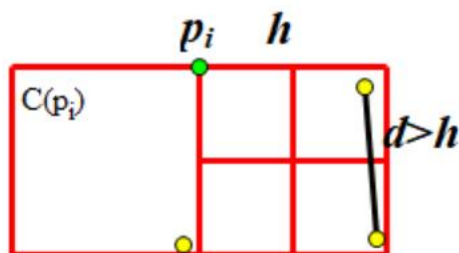
$$C(p_i) = \{p_j \mid p_j \in B, y_i - h < y_j \leq y_i\}$$

在点集 B 中选一点 p_i ，根据 $C(p_i) = \{p_j \mid p_j \in B, y_i - h < y_j \leq y_i\}$ 的规则，得到了由红色方框内的黄色点组成的 C 点集。



按上述过程， $C(p_i)$ 中的所有点的纵坐标都在 $(y_i - h, y_i]$ 范围内；且 $C(p_i)$ 中的所有点，和 p_i 本身，横坐标都在 $(x_m - h, x_m + h)$ 范围内。这构成了一个 $2h \times h$ 的矩形。

我们再将这个矩形拆分为两个 $h \times h$ 的正方形，不考虑 p_i ，其中一个正方形中的点为 $C(p_i) \cap A_1$ ，另一个为 $C(p_i) \cap A_2$ ，且两个正方形内的任意两点间距离大于 h 。（因为它们来自同一下层递归）



我们将一个 $h \times h$ 的正方形拆分为四个 $\frac{h}{2} \times \frac{h}{2}$ 的小正方形。可以发现，每个小正方形中最多有 1 个点：因为该小正方形中任意两点最大距离是对角线的长度，即 $\frac{h}{\sqrt{2}}$ ，该数小于 h 。由此，每个正方形中最多有 4 个点，矩形中最多有 8 个点，去掉 p_i 本身， $\max(C(p_i)) = 7$ 。

1. 排序步骤

在算法的开始阶段，我们需要对点集 P 按照横坐标 x 进行排序。这个步骤的时间复杂度为：

$$O(n \log n)$$

其中 n 是点的数量。

2. 分治步骤

分治步骤中，我们将点集 P 拆分为两个子集 P_1 和 P_2 。在每个递归调用中，问题的规模减半，因此我们可以使用递归关系来描述时间复杂度。

设 $T(n)$ 为处理 n 个点的时间复杂度，则可以写出以下递归关系：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(1)$$

这里的 $O(1)$ 表示处理分治中的常数时间操作（例如计算中间值）。

根据主定理，我们可以解决这个递归关系，得到：

$$T(n) = O(n \log n)$$

3. 合并步骤

合并步骤的复杂度主要体现在以下几个方面：

- **构建集合 B ：**这一操作的复杂度为 $O(n)$ ，因为我们只需要遍历点集 P 一次来选择满足条件的点。
- **排序集合 B ：**在合并步骤中，我们将集合 B 按照纵坐标 y 排序。由于点的数量在最坏情况下仍为 n ，这一操作的复杂度为 $O(n \log n)$ 。
- **计算距离：**对于每个点 p_i 在集合 B 中，我们需要找到与其在集合 $C(p_i)$ 中的点。我们之前已经证明，集合 $C(p_i)$ 中的点数量最多为 7，因此对于每个 p_i ，计算距离的复杂度为常数 $O(1)$ 。

结合这些，我们可以得到合并步骤的复杂度：

$$O(n) + O(n \log n) + O(n) = O(n \log n)$$

综合以上分析，算法的总时间复杂度为：

$$O(n \log n) + O(n \log n) + O(n \log n) = O(n \log n)$$

任务 3：排序算法的实现

一、题目

参照 Insertion 类的实现方式，为其他四个排序算法实现相对应的类类型。这些类类型中有可能需要相配合的成员方法，请同学们灵活处理。针对 Shell 排序，需要分别实现三种不同间隔策略的递减序列（所以 Shell 排序将有三个，分别命名为 Shell1、Shell2 和 Shell3）：

- ① Shell 的最开始的间隔递减序列：1、2、4、...
- ② Hibbard 的间隔递减序列：1、3、7、...、 2^k-1
- ③ Knuth 的间隔递减序列：1、4、13、...、 $(3^k-1)/2$

要求：

- 每个排序算法使用课堂上所讲授的步骤，不要对任何排序算法进行额外的优化；
- 对每个排序算法执行排序之后的数据可以调用 SortAlgorithm 类型中的成员方法 isSorted 进行测试，检查是否排序成功。

二、算法设计

（一）插入排序（Insertion）

插入排序是一种最简单的排序方法，它的基本思想是将一个记录插入到已经排好序的有序表中，从而一个新的、记录数增 1 的有序表。在其实现过程使用双层循环，外层循环对除了第一个元素之外的所有元素，内层循环对当前元素前面有序表进行待插入位置查找，并进行移动。

此算法具有稳定性。

（二）选择排序（Selection）

选择排序的主要流程是：首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置，然后，再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。以此类推，直到所有元素均排序完毕。

此算法不具有稳定性。

（三）快速排序（Quicksort）

快速排序采用的是分治思想，即在一个无序的序列中选取一个任意的基准元素 pivot，利用 pivot 将待排序的序列分成两部分，前面部分元素均小于或等于基准元素，后面部分均大于或等于基准元素，然后采用递归的方法分别对前后两部分重复上述操作，直到将无序序列排列成有序序列。

此算法不具有稳定性。

（四）归并排序（Mergesort）

归并排序是建立在归并操作上的一种有效，稳定的排序算法，该算法是采用分治法的一个非常典型的应用。将已有序的子序列合并，得到完全有序的序列；即先使每个子序列有序，再使子序列段间有序。

此算法具有稳定性。

（五）希尔排序（Shell）

希尔排序是插入排序的一种改进，通过设置间隔 h 来比较和交换元素，然后逐渐缩小间隔，在每个间隔内递归执行插入排序方法。

在本任务中，我们分别采用 Shell1、Hibbard、Knuth 三种间隔策略进行编程。

此算法不具有稳定性。

三、主干代码说明

(一) 插入排序 (Insertion)

```

1 package sort;
2
3 public class Insertion extends SortAlgorithm { 2个用法
4
5     @Override
6     public void sort(Comparable[] objs){
7         int N = objs.length;
8         for(int i = 1; i < N; i++){
9             for(int j = i; j > 0 && less(objs[j], objs[j-1]); j--){
10                 exchange(objs, j, j-1);
11             }
12         }
13     }
14 }

```

(二) 选择排序 (Selection)

```

1 package sort;
2
3 public class Selection extends SortAlgorithm{ 1个用法
4
5     @Override
6     public void sort(Comparable[] objs){
7         for(int i=0;i<objs.length-1;i++){
8             int index=i;
9             for(int j=i+1;j<objs.length;j++){
10                 if(less(objs[j],objs[index])){
11                     index=j;
12                 }
13             }
14             exchange(objs,i,index);
15         }
16     }
17 }

```

(三) 快速排序 (Quicksort)

```

1 package sort;
2
3 public class Quicksort extends SortAlgorithm { 13个用法 3个继承者
4     public int gap; 4个用法
5
6     @Override
7     public void sort(Comparable[] objs){ qsort(objs, 0, objs.length - 1); }
8
9     public void qsort(Comparable[] objs, int i, int j) { 7个用法 2个重写
10         int pivotindex = findpivot(objs, i, j);
11         exchange(objs, pivotindex, j);
12         int k = partition(objs, i - 1, j, objs[j]);
13         exchange(objs, k, j);
14         if ((k - i) > 1) qsort(objs, i, k - 1);
15         if ((j - k) > 1) qsort(objs, k + 1, j);
16     }
17 }

```

```

20 @ public int findpivot(Comparable[] objs, int i, int j) { 4个用法
21     if (less(objs[i], objs[j]) && less(objs[(i + j) / 2], objs[i])) {
22         return i;
23     } else if (less(objs[j], objs[i]) && less(objs[(i + j) / 2], objs[j])) {
24         return j;
25     } else {
26         return (i + j) / 2;
27     }
28 }
29
30
31 @ public int partition(Comparable[] objs, int i, int j, Comparable pivot) { 3个用法
32
33     do {
34         while (less(objs[++i], pivot)) ;
35         while ((j != 0) && (less(pivot, objs[--j]))) ;
36         exchange(objs, i, j);
37     } while (i < j);
38
39     exchange(objs, i, j);
40     return i;
41 }
42 }

```

(四) 归并排序 (Mergesort)

```

1 package sort;
2
3 public class Mergesort extends SortAlgorithm { 1个用法
4
5     @Override
6     @Override public void sort(Comparable[] objs) {
7         Comparable[] temp = new Comparable[objs.length];
8         mergesort(objs, low: 0, high: objs.length-1, temp);
9     }
10
11     public void mergesort(Comparable[] objs, int low, int high, Comparable[] temp) { 3个用法
12         if (low >= high) { return; }
13         int mid = low + (high - low) / 2;
14         mergesort(objs, low, mid, temp);
15         mergesort(objs, low: mid+1, high, temp);
16         merge(objs, low, mid, high, temp);
17     }
18
19     public void merge(Comparable[] objs, int low, int mid, int high, Comparable[] temp) { 1个用法
20         for (int i = low; i <= high; i++) {
21             temp[i] = objs[i];
22         }
23         int i = low;
24         int j = mid + 1;
25         for (int p = low; p <= high; p++) {
26             if (i == mid + 1) {
27                 objs[p] = temp[j++];
28             } else if (j > high) {
29                 objs[p] = temp[i++];
30             } else if (less(temp[j], temp[i])) {
31                 objs[p] = temp[j++];
32             } else {
33                 objs[p] = temp[i++];
34             }
35         }
36     }
37 }
38 }

```

(五) 希尔排序 1 (Shell)

```

1  package sort;
2
3  public class Shell extends SortAlgorithm { 0 个用法
4
5      @Override
6      public void sort(Comparable[] objs) {
7          int N = objs.length;
8          int h = 1;
9          do {
10             h = h * 2;
11         } while (h < N);
12
13         while (h >= 1) {
14             for (int i = h; i < N; i++) {
15                 Comparable temp = objs[i];
16                 int j;
17
18                 for (j = i; j >= h && less(temp, objs[j - h]); j -= h) {
19                     objs[j] = objs[j - h];
20                 }
21
22                 objs[j] = temp;
23             }
24
25             h = h / 2;
26         }
27     }
28 }

```

(六) 希尔排序 2 (Hibbard)

```

1  package sort;
2
3  public class Shell2 extends SortAlgorithm { 0 个用法
4
5      @Override
6      public void sort(Comparable[] objs) {
7          int N = objs.length;
8          int h = 1;
9          do {
10             h = h * 2 + 1;
11         } while (h < N);
12
13         while (h >= 1) {
14             for (int i = h; i < N; i++) {
15                 Comparable temp = objs[i];
16                 int j;
17
18                 for (j = i; j >= h && less(temp, objs[j - h]); j -= h) {
19                     objs[j] = objs[j - h];
20                 }
21
22                 objs[j] = temp;
23             }
24
25             h = (h - 1) / 2;
26         }
27     }
28 }

```

(七) 希尔排序 3 (Knuth)

```
1 package sort;
2
3 public class Shell3 extends SortAlgorithm { 0 个用法
4
5     @Override
6     public void sort(Comparable[] objs) {
7         int N = objs.length;
8         int h = 1;
9         do {
10             h = h * 3 + 1;
11         } while (h < N);
12
13         while (h >= 1) {
14             for (int i = h; i < N; i++) {
15                 Comparable temp = objs[i];
16                 int j;
17
18                 for (j = i; j >= h && less(temp, objs[j - h]); j -= h) {
19                     objs[j] = objs[j - h];
20                 }
21
22                 objs[j] = temp;
23             }
24
25             h = (h - 1) / 3;
26         }
27     }
28 }
```

四、运行结果展示

IsSorted 类代码如下：

```
1 package sort;
2
3 public class IsSorted {
4
5     public static void main(String[] args){
6         SortAlgorithm[] algs = new SortAlgorithm[]{new Insertion(), new Selection(),
7             new Quicksort(), new Mergesort(), new Shell(), new Shell2(), new Shell3()};
8         Double[] arr=GenerateData.getRandomData( N: 1000);
9
10        //分别输出三种排序的结果
11        //插入排序
12        algs[0].sort(arr);
13        System.out.println("插入排序的测试结果为:" + algs[0].isSorted(arr));
14        //选择排序:
15        algs[1].sort(arr);
16        System.out.println("选择排序的测试结果为:" + algs[1].isSorted(arr));
17        //快速排序:
18        algs[2].sort(arr);
19        System.out.println("快速排序的测试结果为:" + algs[2].isSorted(arr));
20        //归并排序:
21        algs[3].sort(arr);
22        System.out.println("归并排序的测试结果为:" + algs[3].isSorted(arr));
23        //Shell策略:
24        algs[4].sort(arr);
25        System.out.println("Shell间隔策略的希尔排序测试结果为:" + algs[4].isSorted(arr));
26        //Hibbard策略:
27        algs[5].sort(arr);
28        System.out.println("Hibbard间隔策略的希尔排序测试结果为:" + algs[5].isSorted(arr));
29    }
30 }
```

```
29         //Knuth策略:
30         algs[6].sort(arr);
31         System.out.println("Knuth间隔策略的希尔排序测试结果为:" + algs[6].isSorted(arr));
32     }
33
34 }
```

运行结果为:

```
D:\IDEA\jdk17\bin\java.exe "-javaagent:D:\IDEA\idea\IntelliJ IDEA 2024.2.4\lib\
插入排序的测试结果为:true
选择排序的测试结果为:true
快速排序的测试结果为:true
归并排序的测试结果为:true
Shell间隔策略的希尔排序测试结果为:true
Hibbard间隔策略的希尔排序测试结果为:true
Knuth间隔策略的希尔排序测试结果为:true

进程已结束，退出代码为 0
```

说明七种排序方法均已成功实现!

任务 4：排序算法性能测试和比较

一、题目

完成对每一个排序算法在数据规模为： 2^8 、 2^9 、 2^{10} 、……、 2^{16} 的均匀分布的随机数据序列、正序序列和逆序序列的排序时间统计。

要求：

- 在同等规模的数据量和数据分布相同下，要做 T 次运行测试，用平均值做为此次测试的结果，用以排除因数据的不同和机器运行当前的状态等因素造成的干扰；（在 SortTest 类型的 test 方法参数中有对每次数据规模下的测试次数的指定）
- 将所有排序算法的运行时间结果用图表的方式进行展示，X 轴代表数据规模，Y 轴代表运行时间。（如果因为算法之间运行时间差异过大而造成显示上的问题，可以通过将运行时间使用取对数的方式调整比例尺）
 - ◆ 注：运行时间会受到机器当时状态的干扰，可能出现不正常的时间趋势表现。如果出现类似现象，可以通过对交换次数与比较次数的统计间接代表算法的运行时间。（比较次数很明确，1 个交换次数可以用 3 个赋值语句进行换算）
- 对实验的结果进行总结：从一个算法的运行时间变化趋势和数据规模的变化角度，从同样的数据规模和相同数据分布下不同算法的时间相对差异上等角度进行阐述。

二、算法设计

使用 LineXY 类进行绘图，将五种排序算法的运行时间与数据规模的关系在同一张图中呈现，X 轴代表数据规模，Y 轴代表运行时间。

在图中绘制七条折线，包括插入排序(Insertion)、选择排序(Selection)、快速排序(Quicksort)、归并排序(Mergesort)以及三种希尔排序(Shell、Shell2、Shell3)。

由于不同算法之间可能会因为运行时间差距过大而造成显示上的问题，可以在纵坐标上取 10 的对数来平衡纵坐标的分布；同时，由于 2 的各次幂之间差距较大，因此横轴数据量取 2 的对数来平衡横坐标的分布。

其中，此处选取 T = 20 作为重复实验的次数。

三、主干代码说明

```
1 package sort;
2
3 import org.jfree.chart.ChartFactory;
4 import org.jfree.chart.ChartPanel;
5 import org.jfree.chart.JFreeChart;
6 import org.jfree.chart.axis.NumberAxis;
7 import org.jfree.chart.plot.PlotOrientation;
8 import org.jfree.chart.plot.XYPlot;
9 import org.jfree.chart.renderer.xy.XYLineAndShapeRenderer;
10 import org.jfree.chart.ui.ApplicationFrame;
11 import org.jfree.chart.ui.RectangleInsets;
12 import org.jfree.data.xy.XYDataset;
13 import org.jfree.data.xy.XYSeries;
14 import org.jfree.data.xy.XYSeriesCollection;
15
16 import java.awt.*;
17
18 public class LineXYDemo extends ApplicationFrame {
19     // 该构造方法中完成了数据集、图表对象和显示图表面板的创建工作
20     public LineXYDemo(String title, SortAlgorithm[] algorithms, int[] dataSizes, int testIterations) { 0 个用法
21         super(title);
22         XYDataset dataset = createDataset(algorithms, dataSizes, testIterations); // 创建记录图中坐标点的数据集
23         JFreeChart chart = createChart(dataset); // 使用上一步已经创建好的数据集生成一个图表对象
24         ChartPanel chartPanel = new ChartPanel(chart); // 将上一步已经创建好的图表对象放置到一个可以显示的Panel上
25         // 设置GUI面板Panel的显示大小
26         chartPanel.setPreferredSize(new Dimension(1500, 500));
27         setContentPane(chartPanel); // 这是JavaGUI的步骤之一，不用过于关心，面向对象课程综合训练的视频中进行了讲解。
28     }
```

```

50 @ private JFreeChart createChart(XYDataset dataset) { 2个用法
51     // 使用已经创建好的dataset生成图表对象
52     // JFreeChart提供了多种类型的图表对象，本次实验是需要使用XYLine型的图表对象
53     JFreeChart chart = ChartFactory.createXYLineChart(
54         title: "Sorting Algorithm Performance", // 图表的标题
55         "Data Size", // 横轴的标题名
56         "Time (nanoseconds)", // 纵轴的标题名
57         dataset, // 图表对象中使用的数据集对象
58         PlotOrientation.VERTICAL, // 图表显示的方向
59         true, // 是否显示图例
60         false, // 是否需要生成tooltips
61         false // 是否需要生成urls
62     );
63     // 下面所做的工作都是可选操作，主要是为了调整图表显示的风格
64     // 同学们不必在意下面的代码
65     // 可以将下面的代码去掉对比一下显示的不同效果
66     chart.setBackgroundPaint(Color.WHITE);
67     XYPlot plot = (XYPlot) chart.getPlot();
68
69     NumberAxis rangeAxis = (NumberAxis) plot.getRangeAxis();
70     rangeAxis.setRange(lower: 4, rangeAxis.getUpperBound()); // 设置纵轴的起始值为 4.0
71
72
73     plot.setBackgroundPaint(Color.LightGray);
74     plot.setAxisOffset(new RectangleInsets(top: 5.0, left: 5.0, bottom: 5.0, right: 6.0));
75     plot.setDomainGridlinePaint(Color.WHITE);
76     plot.setRangeGridlinePaint(Color.WHITE);
77     XYLineAndShapeRenderer renderer = (XYLineAndShapeRenderer) plot.getRenderer();
78     renderer.setDefaultShapesVisible(true);
79     renderer.setDefaultShapesFilled(true);
80     return chart;
81 }

```

```

166 @ private XYDataset createDataset(SortAlgorithm[] algorithms, int[] dataSizes, int testIterations) { 1个用法
167     XYSeriesCollection dataset = new XYSeriesCollection();
168
169     for (SortAlgorithm algorithm : algorithms) {
170         XYSeries series = new XYSeries(algorithm.getClass().getSimpleName());
171         for (int dataSize : dataSizes) {
172             Double[] data = GenerateData.getSortedData(dataSize);
173             double averageTime = (SortTest.testForAscending(algorithm, dataSize, testIterations));
174             //将横坐标取二的对数间隔 (log2)；纵坐标作十的对数间隔处理 (lg)
175             series.add(⊗ Math.log(dataSize) / Math.log(2), Math.log10(averageTime));
176         }
177         dataset.addSeries(series);
178     }
179
180     return dataset;
181 }

```

```

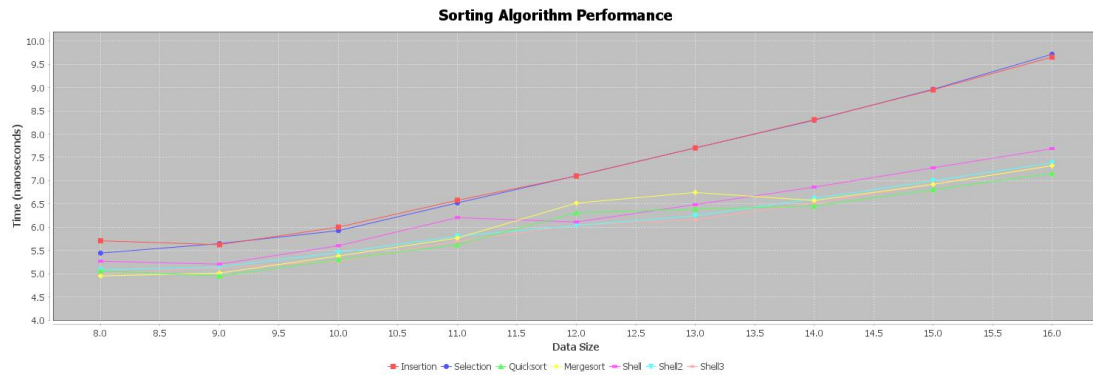
200 ▶ public static void main(String[] args) {
201     //绘制七条图线，包括插入排序、选择排序、快速排序、归并排序与Shell、Hibbard、Knuth三种版本的希尔排序
202     SortAlgorithm[] algorithms = {new Insertion(), new Selection(), new Quicksort(),
203         new Mergesort(), new Shell(), new Shell2(), new Shell3()};
204     //横坐标取2的八次方到16次方共9个数
205     int[] dataSizes = new int[9];
206     for (int i = 8; i <= 16; i++) {
207         dataSizes[i - 8] = (int) Math.pow(2, i);
208     }
209     int testIterations = 20;
210     LineXYDemo demo = new LineXYDemo(title: "Sorting Algorithm Performance", algorithms, dataSizes, testIterations);
211     demo.pack();
212     demo.setVisible(true);
213 }

```

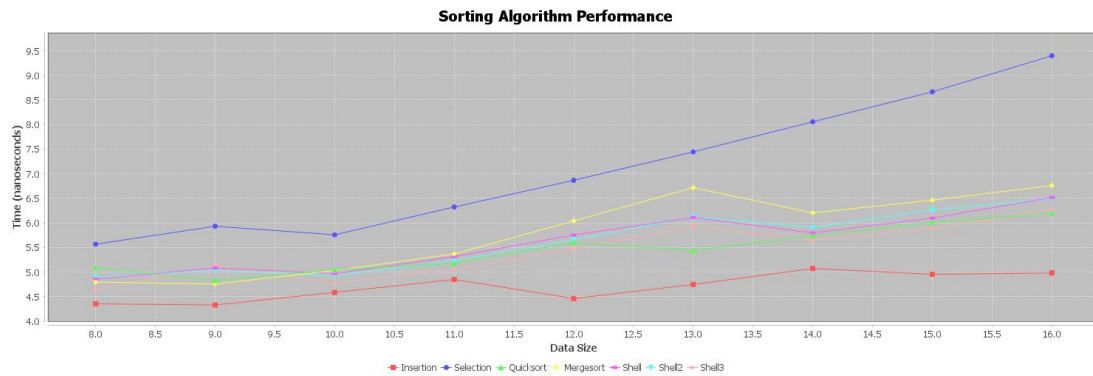
四、运行结果展示

下列三张图为上一任务中七种排序算法分别在数据规模为 $2^8, 2^9, \dots, 2^{16}$ 上的均匀分布的随机数据序列、正序序列和逆序序列的排序时间统计：

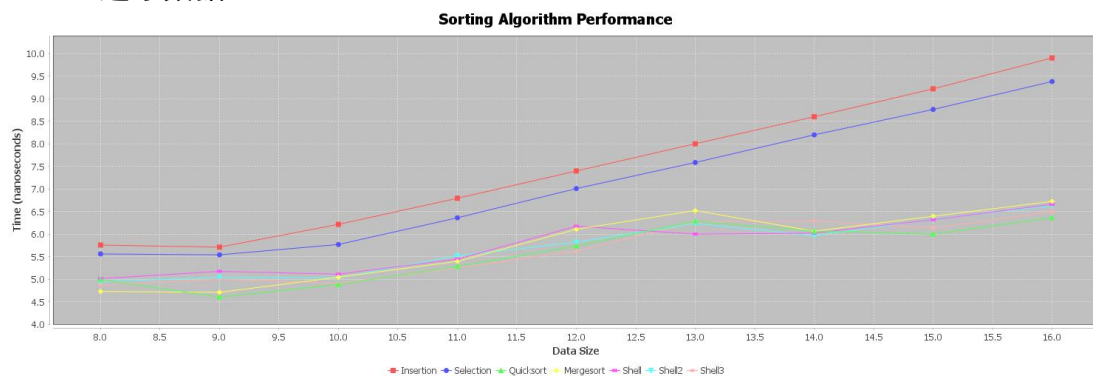
（一）均匀分布的随机数据：



（二）正序数据：



（三）逆序数据：



五、对结果的分析

（一）从一个算法的运行时间变化趋势和数据规模的变化角度：

①插入排序（Insertion）

插入排序在小规模数据上表现较好，其运行时间随着数据规模的增加呈现二次增长趋势。

因为需要大量的比较和移动操作，插入排序在正序数据上表现最好，而在逆序数据上表现最差。

②选择排序（Selection）

选择排序的运行时间随着数据规模的增加呈现明显的二次增长趋势，性能较差。

无论在规模多大的数据集上，因为要进行大量的比较和交换，选择排序的表现均是七种排序中最差的。

③快速排序（Quicksort）

随着数据规模的增加，快速排序表现出明显的优势，运行时间呈现近似线性的增长趋势。

此外，在不同数据分布下，对于随机数据、正序数据和逆序数据，快速排序的表现相对一致，这展现了它的普适性与高效性。

④归并排序（Mergesort）

归并排序的运行时间随着数据规模的增加呈现近似线性的增长趋势，性能相对较好。

此外，在不同数据分布下，对于随机数据、正序数据和逆序数据，归并排序的表现相对一致，这展现了它的普适性与高效性。

特别地，其在较小、较大数据集上优势明显，但在中等规模数据集上优势不明显。

⑤希尔排序（Shell）

希尔排序的运行时间随着数据规模的增加呈现复杂的趋势，这取决于间隔递减序列的选择。从总体上看，三种希尔排序的性能顺序为：Knuth > Hibbard > Shell

在大规模数据上，希尔排序的性能相对较好，尤其在适当选择间隔递减序列的情况下。

结论：

在大规模数据集上，快速排序和希尔排序通常是较为优秀的选择，因为它们的运行时间呈线性或近似线性增长。

在小规模数据集上，插入排序和选择排序可能更适用，因为它们在较小规模上的性能相对较好。

归并排序在较小、较大规模数据集上的性能具有一定优势，但在中等规模数据上不如快速排序与希尔排序，同时，需要注意递减序列的选择。

（二）从同样的数据规模和相同数据分布下不同算法的时间相对差异：

①均匀分布的随机数据

快速排序与希尔排序在处理大规模随机数据时表现良好，因为它们平均时间复杂度较低。

希尔排序在中等规模的数据集上表现不错，对于其间隔递减序列的选取可以提升排序效率。因为时间复杂度较高，选择排序和插入排序在大规模数据上相对较慢。

②正序数据

插入排序在正序数据上表现较好，因为它只需要少量的比较和移动操作。

希尔排序的间隔序列在正序数据上的变化趋势与随机数据大体相同，在中等规模数据集上仍然比较高效。

③逆序数据

因为需要大量的比较和移动操作，**插入排序**和**选择排序**在此数据集上运行时间最长。

快速排序和**希尔排序**在逆序数据上表现相对较好，尤其是**归并排序**的分治法对逆序数据具有较好的适应性。

结论：

在大规模数据集上，**希尔排序**和**快速排序**效果相差不大。

插入排序和**选择排序**适用于小规模数据，但在大规模数据集上性能较差。

归并排序在较小与较大规模数据集上性能与快速排序与希尔排序较为接近，甚至性能优于二者，但在中等规模数据集上不占优势。

六、总结与收获

通过对七种不同排序算法在不同数据规模、分布下的运行时间统计，可以注意到，随着数据规模增加，**快速排序**和**希尔排序**表现出较为线性的增长趋势，尤其在大规模数据上；相比之下，**插入排序**和**选择排序**在小规模数据上表现相对较好；而**归并排序**则在较小与较大规模数据上有一定优势，中等规模反而不占优势；任务中提到的三种希尔排序增长趋势大致相同，性能顺序为：Knuth > Hibbard > Shell。

本任务深化了我对此七种排序算法性能与数据规模与分布之间关系的理解，使我对算法的性能有了更直观的感觉，为我未来在学习工作中选择与运用合适的算法提供了参考与经验。

任务 5：快速排序的再探讨与应用

一、题目

快速排序算法被誉为 20 世纪科学和工程领域的十大算法之一。前面的任务只是对快速排序的初识，下面从几个方面再更深入了解它：

- ① 探索快速排序的递归深度：为快速排序算法增加一个整数类型的参数 `depth`，每递归调用一次让 `depth` 加 1，每从递归中返回就让 `depth` 减 1，在整个执行过程中，`depth` 的最大值即为快速排序的递归深度，验证在平均情形下，`depth` 的值和规模 `N` 之间的关系；
- ② 优化快速排序：当待排序的数据量小于某个阈值时将递归的快速排序调用改为直接插入排序调用，按照这种策略的优化的快速排序算法参照任务 4 的要求进行测试，并与任务 4 中没有优化的快速排序算法的执行时间进行对比；
- ③ 螺母与螺丝的匹配问题：给定一堆螺丝和螺母，它们具有不同的尺寸，且螺丝和螺母之间存在一对一匹配关系，但不能直接比较螺丝或螺母的尺寸大小，只能通过给定的 `match(nut bolt)` 函数进行比较，该函数返回 0 表示匹配，1 表示螺母尺寸大于螺钉尺寸，-1 表示螺母尺寸小于螺钉尺寸。设计一个算法实现对螺母与螺丝的匹配，给出算法的时间复杂度分析。可以通过设置两个数组 `nuts` 和 `bolts`，数组中的内容是它们各自的大小（保证大小各不相同且 `nuts` 和 `bolts` 数组中的数据内容完全一致），编写程序测试你所设计的算法是否正确。

二、数据结构设计

数据规模：

选用的数据规模为 $2^8 - 2^{16}$

数据分布：

根据任务要求，有三种数据分布：均匀分布的随机数据、正序数据和逆序数据。

用 `N` 来表示数据规模的大小。

对于均匀分布的随机数据，可以使用 `GenerateData.getRandomData(N)` 方法生成。

对于正序数据，可以使用 `GenerateData.getSortedData(N)` 方法生成。

对于逆序数据，可以使用 `GenerateData.getInversedData(N)` 方法生成。

此处我们沿用上一任务中的设定，取 `T = 20`。

三、算法设计

（一）探索递归深度与数据规模的关系

略微修改上一任务中一般的快速排序算法，为其添加了 `getMaxDepth()` 方法。观察在各个数据规模下，分别多次使用快速排序算法，统计递归深度的平均值并绘制折线图。

（二）优化快速排序（Optimized QuickSort）

整体上仍采用分治策略，使用经典的快速排序算法，当数据规模小于某个阈值时，切换为直接插入排序，以减少递归深度。在实验中，通过调整阈值，观察对快速排序算法性能的影响。

（三）螺丝螺母匹配问题

1、算法思想：

算法采用了类似快速排序的分治策略：通过选择一个螺母作为基准，将螺丝数组分成三部分：小于、等于和大于该螺母的螺丝。接着，使用选择出的螺丝对螺母进行相同的分区。通过递归处理子数组，最终可以实现螺母和螺丝的完全匹配。

2、具体实现：

match 函数：模拟螺母和螺丝的比较。

matchNutsAndBolts 函数：主匹配函数，接受螺母和螺丝数组及其边界，进行递归处理。

partition 函数：用于根据基准元素将数组划分为小于、等于和大于基准元素的三个部分。

实现步骤：

- 1) 选择基准：选择 nuts[low]作为螺母的基准。
- 2) 划分螺丝：通过 partition 函数将螺丝数组划分，将小于、等于、大于基准的螺丝分开。
- 3) 划分螺母：使用找到的螺丝对螺母数组进行相同的划分。
- 4) 递归处理：对划分后的左右子数组进行递归匹配，直到所有螺母和螺丝都被处理完。

3、时间复杂度分析：

该算法的时间复杂度与快速排序相似。

最优情况：在每次划分时，所选基准元恰能将数组平衡划分。此时，时间复杂度的递推公式为：

$$T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$$

最差情况：当基准元素总是选择到最小或最大元素时，划分会极不平衡，导致递归深度为 $O(n)$ 。此时，时间复杂度的递推方程为：

$$T(n) = T(n-1) + O(n) \Rightarrow T(n) = O(n^2)$$

空间复杂度：算法的空间复杂度主要由递归栈的深度决定，最坏情况下为 $O(n)$ ，最优情况下为 $O(\log n)$ 。

四、主干代码说明

（一）探寻递归深度与数据规模的关系

①快速排序算法（计算递归深度版本）

```
1 package task5;
2
3 public class Quicksort extends SortAlgorithm { 10个用法
4     public int gap; 1个用法
5     private int depth; // 用于追踪递归深度 6个用法
6     private int maxDepth; // 记录最大递归深度 5个用法
7
8     public void sort(Comparable[] objs) {
9         depth = 0; // 初始化递归深度
10        maxDepth = 0; // 初始化最大递归深度
11        qsort(objs, 0, objs.length - 1);
12    }
```

```

14     public void qsort(Comparable[] objs, int i, int j) { 3个用法
15
16         depth++; // 进入递归, 深度加 1
17         if (depth > maxDepth) {
18             maxDepth = depth; // 更新最大递归深度
19         }
20         System.out.println("Current depth: " + depth); // 打印当前深度
21
22         int pivotindex = findpivot(objs, i, j);
23         exchange(objs, pivotindex, j);
24         int k = partition(objs, i - 1, j, objs[j]);
25         exchange(objs, k, j);
26
27         if ((k - i) > 1) qsort(objs, i, k - 1);
28         if ((j - k) > 1) qsort(objs, k + 1, j);
29
30         depth--; // 递归返回, 深度减 1
31     }

```

```

33 @    public int findpivot(Comparable[] objs, int i, int j) { 1个用法
34         if (less(objs[i], objs[j]) && less(objs[(i + j) / 2], objs[i])) {
35             return i;
36         } else if (less(objs[j], objs[i]) && less(objs[(i + j) / 2], objs[j])) {
37             return j;
38         } else {
39             return (i + j) / 2;
40         }
41     }
42
43 @    public int partition(Comparable[] objs, int i, int j, Comparable pivot) { 1个用法
44         do {
45             while (less(objs[++i], pivot));
46             while ((j != 0) && (less(pivot, objs[--j])));
47             exchange(objs, i, j);
48         } while (i < j);
49
50         exchange(objs, i, j);
51         return i;
52     }
53
54     // 返回最大递归深度
55     public int getMaxDepth() { 3个用法
56         return maxDepth; // 返回最大深度
57     }
58
59     // 重置递归深度
60     public void resetDepth() { 2个用法
61         maxDepth = 0; // 返回最大深度
62     }
63 }

```

②函数图象（以随机数据为例）

```

1     package task5;
2
3     import org.jfree.chart.ChartFactory;
4     import org.jfree.chart.ChartPanel;
5     import org.jfree.chart.JFreeChart;
6     import org.jfree.chart.axis.NumberAxis;
7     import org.jfree.chart.plot.PlotOrientation;
8     import org.jfree.chart.plot.XYPlot;
9     import org.jfree.chart.renderer.xy.XYLineAndShapeRenderer;
10    import org.jfree.chart.ui.ApplicationFrame;
11    import org.jfree.chart.ui.RectangleInsets;
12    import org.jfree.data.xy.XYDataset;
13    import org.jfree.data.xy.XYSeries;
14    import org.jfree.data.xy.XYSeriesCollection;
15    import java.awt.*;
16
17    public class LineXYDemo2 extends ApplicationFrame {
18        public LineXYDemo2(String title, task5.QuickSort[] algorithms, int[] dataSizes, int testIterations) { 1个用法
19            super(title);
20            XYDataset dataset = createDatasetForDepth(algorithms, dataSizes, testIterations);
21            JFreeChart chart = createChartForDepth(dataset);
22            ChartPanel chartPanel = new ChartPanel(chart);
23            chartPanel.setPreferredSize(new Dimension(1500, 500));
24            setContentPane(chartPanel);
25        }

```

```

27  @ private JFreeChart createChartForDepth(XYDataset dataset) { 1个用法
28      JFreeChart chart = ChartFactory.createXYLineChart(
29          title: "Quicksort Recursive Depth",
30          xAxisLabel: "log2(N)",
31          yAxisLabel: "Depth",
32          dataset,
33          PlotOrientation.VERTICAL,
34          legend: true,
35          tooltips: false,
36          urls: false
37      );
38
39      chart.setBackgroundPaint(Color.WHITE);
40      XYPlot plot = (XYPlot) chart.getPlot();
41      NumberAxis rangeAxis = (NumberAxis) plot.getRangeAxis();
42      rangeAxis.setRange(lower: 0, rangeAxis.getUpperBound());
43
44      plot.setBackgroundPaint(Color.LightGray);
45      plot.setAxisOffset(new RectangleInsets(top: 5.0, left: 5.0, bottom: 5.0, right: 6.0));
46      plot.setDomainGridlinePaint(Color.WHITE);
47      plot.setRangeGridlinePaint(Color.WHITE);
48      XYLineAndShapeRenderer renderer = (XYLineAndShapeRenderer) plot.getRenderer();
49      renderer.setDefaultShapesVisible(true);
50      renderer.setDefaultShapesFilled(true);
51      return chart;
52  }

```

```

54  @ private XYDataset createDatasetForDepth(task5.Quicksort[] algorithms, int[] dataSizes, int testIterations) { 1个用法
55      XYSeriesCollection dataset = new XYSeriesCollection();
56
57      for (task5.Quicksort algorithm : algorithms) {
58          XYSeries series = new XYSeries(key: algorithm.getClass().getSimpleName() + algorithm.gap);
59          for (int dataSize : dataSizes) {
60              double maxDepth = getMaxDepth(algorithm, dataSize, testIterations);
61              series.add(x: Math.log(dataSize) / Math.log(2), maxDepth);
62          }
63          dataset.addSeries(series);
64      }
65
66      return dataset;
67  }
68
69  private double getMaxDepth(task5.Quicksort algorithm, int dataSize, int testIterations) { 1个用法
70      double maxDepth = 0;
71      for (int i = 0; i < testIterations; i++) {
72          Comparable[] array = GenerateData.getRandomData(dataSize);
73          algorithm.sort(array);
74          int depth = algorithm.getMaxDepth();
75          if (depth > maxDepth) {
76              maxDepth = depth;
77          }
78          algorithm.resetDepth();
79      }
80      return maxDepth;
81  }

```

```

83  public static void main(String[] args) {
84      Quicksort[] algorithms = {new Quicksort()};
85
86      int[] dataSizes = new int[9];
87      for (int i = 8; i <= 16; i++) {
88          dataSizes[i - 8] = (int) Math.pow(2, i);
89      }
90
91      int testIterations = 20;
92      LineXYDemo2 demo = new LineXYDemo2(title: "Quicksort Recursive Depth", algorithms, dataSizes, testIterations);
93      demo.pack();
94      demo.setVisible(true);
95  }
96  }

```

（二）优化快速排序（Optimized QuickSort）

①优化快速排序算法（QuickSortO）

```
1 package sort;
2
3 public class Quicksort0 extends Quicksort{ 3个用法 1个继承者
4
5     //优化快速排序算法的阈值
6     public Quicksort0(int gap){ this.gap = gap; }
7
8
9
10 public void qsort(Comparable[] objs, int i, int j){ 7个用法 1个重写
11     if(j - i < 3){
12         insertsort(objs,i,j);
13     }else {
14         int pivoindex = findpivot(objs,i,j);
15         exchange(objs,pivoindex,j);
16         int k = partition(objs, i-1,j,objs[j]);
17         exchange(objs,k,j);
18         if((k - i) > 1)qsort(objs,i, k-1);
19         if((j - k) > 1)qsort(objs, k+1,j);}
20     }
21
22
23     public void insertsort(Comparable[] objs,int low,int high){ 2个用法
24         for(int i = low + 1; i <= high; i++){
25             for(int j = i; j > low && less(objs[j], objs[j-1]); j--){
26                 exchange(objs, j, j-1);
27             }
28         }
29     }
30 }
```

②画图（以随机数据为例）

```
1 package task5;
2
3 import org.jfree.chart.ChartFactory;
4 import org.jfree.chart.ChartPanel;
5 import org.jfree.chart.JFreeChart;
6 import org.jfree.chart.axis.NumberAxis;
7 import org.jfree.chart.plot.PlotOrientation;
8 import org.jfree.chart.plot.XYPlot;
9 import org.jfree.chart.renderer.xy.XYLineAndShapeRenderer;
10 import org.jfree.chart.ui.ApplicationFrame;
11 import org.jfree.chart.ui.RectangleInsets;
12 import org.jfree.data.xy.XYDataset;
13 import org.jfree.data.xy.XYSeries;
14 import org.jfree.data.xy.XYSeriesCollection;
15 import sort.Quicksort;
16 import sort.Quicksort0;
17 import sort.*;
18
19 import java.awt.*;
20
21 public class LineXYDemo extends ApplicationFrame {
22     // 该构造方法中完成了数据集、图表对象和显示图表面板的创建工作
23     public LineXYDemo(String title, sort.Quicksort[] algorithms, int[] dataSizes, int testIterations) { 1个用法
24         super(title);
25         XYDataset dataset = createDatasetNewQuick(algorithms, dataSizes, testIterations); // 创建记录图中坐标点的数据集
26         JFreeChart chart = createChart(dataset); // 使用上一步已经创建好的数据集生成一个图表对象
27         ChartPanel chartPanel = new ChartPanel(chart); // 将上一步已经创建好的图表对象放置到一个可以显示的Panel上
28         // 设置GUI面板Panel的显示大小
29         chartPanel.setPreferredSize(new Dimension(1500, 500));
30         setContentPane(chartPanel); // 这是JavaGUI的步骤之一，不用过于关心，面向对象课程综合训练的视频中进行了讲解。
31     }
```

```
33 private JFreeChart createChart(XYDataset dataset) { 1个用法
34     // 使用已经创建好的dataset生成图表对象
35     // JFreeChart提供了多种类型的图表对象，本次实验是需要使用XYLine型的图表对象
36     JFreeChart chart = ChartFactory.createXYLineChart(
37         "Sorting Algorithm Performance", // 图表的标题
38         "Data Size", // 横轴的标题名
39         "Time (nanoseconds)", // 纵轴的标题名
40         dataset, // 图表对象中使用的数据集对象
41         PlotOrientation.VERTICAL, // 图表显示的方向
42         true, // 是否显示图例
43         false, // 是否需要生成tooltips
44         false // 是否需要生成URLs
45     );
```

```

46 // 下面所做的工作都是可选操作，主要是为了调整图表显示的风格
47 // 同学们不必在意下面的代码
48 // 可以将下面的代码去掉对比一下显示的不同效果
49 chart.setBackgroundPaint(Color.WHITE);
50 XYPlot plot = (XYPlot) chart.getPlot();
51
52 NumberAxis rangeAxis = (NumberAxis) plot.getRangeAxis();
53 rangeAxis.setRange( lower: 4, rangeAxis.getUpperBound()); // 设置纵轴的起始值为 4.0
54
55
56 plot.setBackgroundPaint(Color.LightGray);
57 plot.setAxisOffset(new RectangleInsets( top: 5.0, left: 5.0, bottom: 5.0, right: 6.0));
58 plot.setDomainGridlinePaint(Color.WHITE);
59 plot.setRangeGridlinePaint(Color.WHITE);
60 XYLineAndShapeRenderer renderer = (XYLineAndShapeRenderer) plot.getRenderer();
61 renderer.setDefaultShapesVisible(true);
62 renderer.setDefaultShapesFilled(true);
63 return chart;
64 }

```

```

66 @ private XYDataset createDatasetNewQuick(sort.QuickSort[] algorithms, int[] dataSizes, int testIterations) { 1个用法
67     XYSeriesCollection dataset = new XYSeriesCollection();
68
69     for (sort.QuickSort algorithm : algorithms) {
70         XYSeries series = new XYSeries( key: algorithm.getClass().getSimpleName() + algorithm.gap);
71         for (int dataSize : dataSizes) {
72             double averageTime = (SortTest.testForRandom(algorithm, dataSize, testIterations));
73             //将横坐标取 log2; 纵坐标取 lg
74             series.add( x: Math.log(dataSize) / Math.log(2), Math.log10(averageTime));
75         }
76         dataset.addSeries(series);
77     }
78
79     return dataset;
80 }

```

③主函数代码

```

82 public static void main(String[] args) {
83     //绘制五条图线，包括未优化 与 阈值为3,7,15,31的优化快速排序算法
84     QuickSort[] algorithms = new QuickSort[5];
85     algorithms[0] = new QuickSort();
86     int j = 1;
87     for (int i = 2; i <= 5; i++) {
88         algorithms[j++] = new QuickSort( gap: (int) Math.pow(2, i) - 1);
89     }
90
91     //横坐标取2的八次方到16次方共9个数
92     int[] dataSizes = new int[9];
93     for (int i = 8; i <= 16; i++) {
94         dataSizes[i - 8] = (int) Math.pow(2, i);
95     }
96
97     int testIterations = 20;
98     LineXYDemo demo = new LineXYDemo( title: "Sorting Algorithm Performance", algorithms, dataSizes, testIterations);
99     demo.pack();
100     demo.setVisible(true);
101 }
102
103 }

```

(三) 螺丝螺母匹配问题

①算法代码

```

26 private static int partition(int[] arr, int low, int high, int pivot) { 2个用法
27     int i = low;
28     for (int j = low; j <= high; j++) {
29         int compareResult = match(arr[j], pivot);
30         if (compareResult == -1) {
31             // 交换元素
32             int temp = arr[i];
33             arr[i] = arr[j];
34             arr[j] = temp;
35             i++;
36         }
37     }

```

```

38 // 将 pivot 放到正确位置
39 for (int k = low; k <= high; k++) {
40     if (match(arr[k], pivot) == 0) {
41         int temp = arr[k];
42         arr[k] = arr[i];
43         arr[i] = temp;
44         break;
45     }
46 }
47 return i;
48 }

```

②检测算法的正确性

```

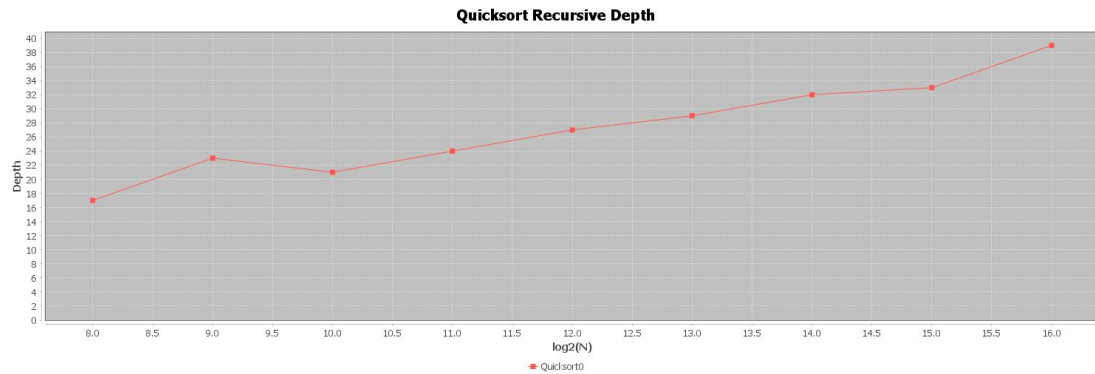
50 public static void main(String[] args) {
51     int[] nuts = {6, 5, 3, 1, 4, 2};
52     int[] bolts = {2, 4, 6, 1, 3, 5};
53
54     System.out.println("Before matching:");
55     System.out.println("Nuts: " + Arrays.toString(nuts));
56     System.out.println("Bolts: " + Arrays.toString(bolts));
57
58     matchNutsAndBolts(nuts, bolts, low: 0, high: nuts.length - 1);
59
60     System.out.println("\nAfter matching:");
61     System.out.println("Nuts: " + Arrays.toString(nuts));
62     System.out.println("Bolts: " + Arrays.toString(bolts));
63 }
64 }

```

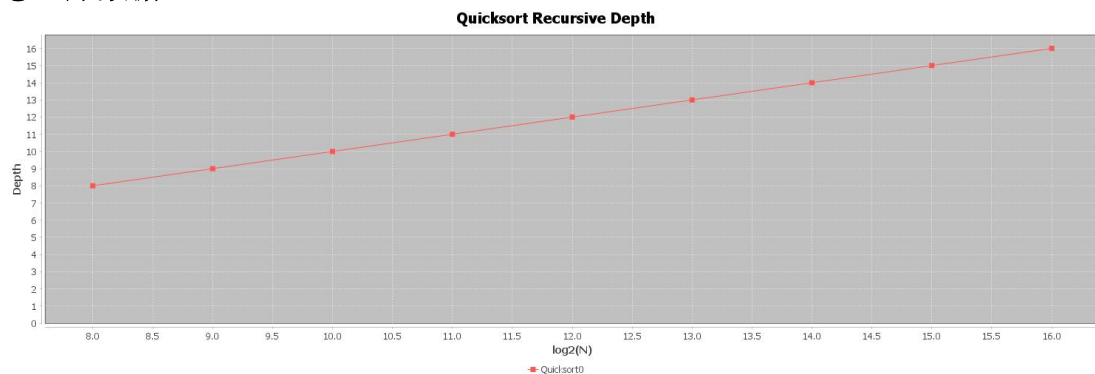
五、运行结果展示

（一）递归深度与数据集规模的关系

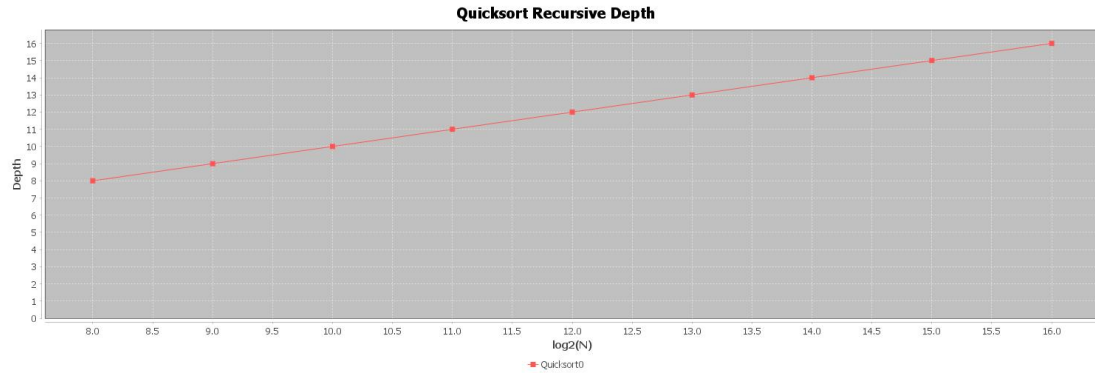
①分布均匀的随机数据



②正序数据

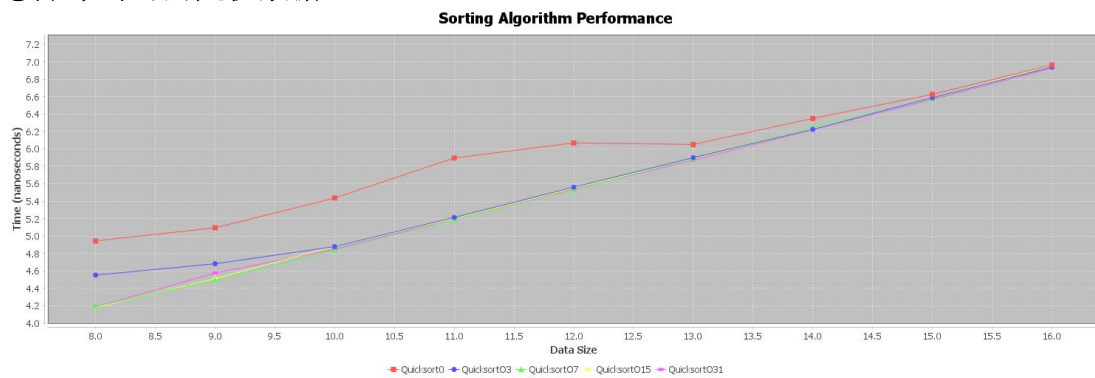


③逆序数据

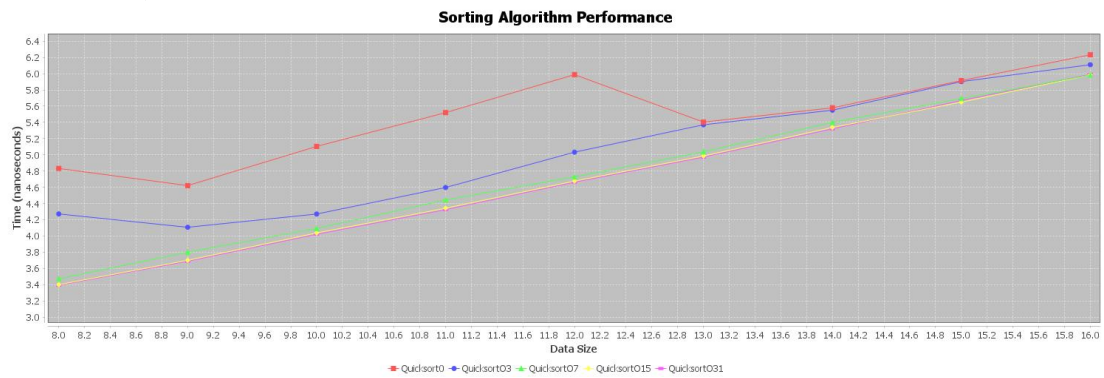


(二) 优化快速排序 (Optimized QuickSort)

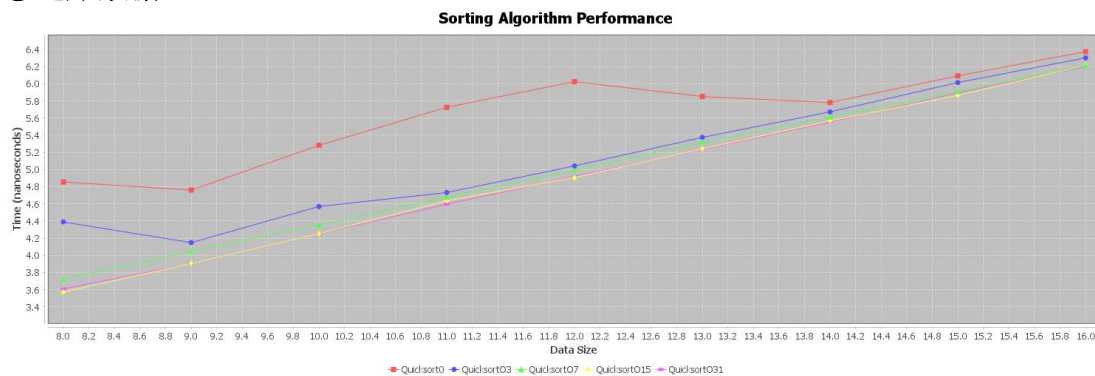
①分布均匀的随机数据



②正序数据



③逆序数据



（三）螺丝螺母匹配问题

①检验算法的正确性

```
D:\IDEA\jdk17\bin\java.exe "-javaagent:D:\IDEA\idea\IntelliJ IDEA 2024.2.4\lib\
Before matching:
Nuts: [6, 5, 3, 1, 4, 2]
Bolts: [2, 4, 6, 1, 3, 5]

After matching:
Nuts: [1, 2, 3, 4, 5, 6]
Bolts: [1, 2, 3, 4, 5, 6]

进程已结束，退出代码为 0
```

六、总结与收获

（一）探索递归深度与数据规模的关系

通过图像中我们可以注意到，随着数据规模的增加，快速排序算法的递归深度随横坐标的增加呈明显的线性增长趋势。由此我们可以得出，对于一个一般的规模为 n 的问题，快速排序算法的递归深度大致为 $O(\log n)$ 的速率增长的。

（二）优化快速排序（Optimized QuickSort）

我们可以直观地在运行时间的图像中观察到，无论数据序列是随机、正序还是逆序，引入直接插入排序优化后的 QuickSortO 无论取任一种阈值，相比未优化的 QuickSort 都有明显提升。

针对不同的数据量，阈值的选择对算法的提升程度具有较强的相关性：通过引入直接插入排序使得快速排序的递归深度减小，从而减少了递归过程中的一些开销。这通常在处理较小规模的数据时会带来性能提升，因为直接插入排序在小规模数据上的表现可能比快速排序更好。优化程度好坏，阈值的选择是关键，过小的阈值可能导致频繁的切换，而过大的阈值可能无法充分发挥插入排序的优势。

特别地，我们从图中可以发现，阈值取为 15 的 QuickSortO 算法在一般情形下表现最为优异。

（三）螺丝螺母匹配问题

这个应用问题使我更深入地理解了快速排序的原理和应用，并提高了对快速排序算法的掌握水平。

（四）总结

在本任务中，我学到了关于快速排序算法的更深入的知识。通过引入不同的阈值进行优化，当数据规模较小时，转而使用直接插入排序，有效减小递归深度，提高了排序算法的性能。此外，我还学会了使用快速排序算法的思想来解决问题现实问题，这给我带来了一定的启发。